

# Knowledge Organiser — 1.3 Exchanging Data

OCR A Level Computer Science (H446) — Component 01, Section 1.3 (Compression, encryption & hashing; Databases; Networks; Web technologies)

## 1.3.1 Compression, Encryption & Hashing

**Compression** = reduce file size → less storage, less bandwidth, faster transfer.

|                               | Lossless             | Lossy                         |
|-------------------------------|----------------------|-------------------------------|
| Data removed permanently?     | No                   | Yes                           |
| Original recoverable exactly? | Yes                  | No                            |
| Size reduction                | Smaller              | Larger                        |
| Uses                          | Text, code, ZIP, PNG | JPEG images, MP3 audio, video |

- **Run Length Encoding (RLE):** lossless; replaces *runs* of repeated values with one value + a count. Best for long runs (simple images); can *enlarge* files with few repeats.
- **Dictionary compression:** lossless; stores recurring patterns once in a *dictionary* (index), replacing each with a short reference. Dictionary must be sent with the file. Good for text (e.g. LZW).

**Encryption** = scramble plaintext → ciphertext using algorithm + key; protects confidentiality if intercepted.

|               | Symmetric                    | Asymmetric                                     |
|---------------|------------------------------|------------------------------------------------|
| Keys          | Same key encrypts & decrypts | Public + private key pair                      |
| Speed         | Fast                         | Slower                                         |
| Issue/benefit | Key distribution problem     | Public key shared openly — solves distribution |

- Asymmetric: data encrypted with **public** key → only matching **private** key decrypts. Enables digital signatures (sign with private, verify with public). HTTPS uses asymmetric to swap a symmetric session key, then symmetric for bulk data.

**Hashing:** hash function → fixed-length **hash/digest**. Properties: **one-way** (not reversible), **deterministic**, few **collisions**, fast. *Uses:* storing passwords (store hash, not password); hash tables ( $\approx O(1)$  lookup). Hashing  $\neq$  encryption (encryption is reversible with a key).

## 1.3.2 Databases

| Flat file                        | Relational                |
|----------------------------------|---------------------------|
| Single table                     | Multiple linked tables    |
| High redundancy                  | Reduced redundancy        |
| Prone to anomalies/inconsistency | Consistent (normalised)   |
| Simple                           | Scalable, complex queries |

- **Entity** = thing data is stored about (→ table). **Attribute/field** = column. **Record/row/tuple** = one instance.

**Keys** | Key | Meaning | |---|---| | Primary | Attribute(s) uniquely identifying each record | | Foreign | Primary key of *another* table, used to link tables | | Composite | Primary key made of 2+ attributes combined | | Secondary | Non-primary attribute used to search/index quickly |

- **ER modelling:** relationships are 1:1, 1:M, M:N. A **many-to-many** is resolved with a **linking/junction table** (composite key).
- **Referential integrity:** a foreign key must always reference a valid existing primary key (or be null) → no orphaned records.

**Normalisation** = organise data to reduce redundancy & remove anomalies. | Form | Rule (in addition to previous) | |---|---| | 1NF | No repeating groups; atomic values; has a primary key | | 2NF | No **partial** dependencies (every non-key attribute depends on the *whole* key) | | 3NF | No **transitive** dependencies (no non-key attribute depends on another non-key) |

*"The key, the whole key, and nothing but the key."*

**SQL keywords** | Keyword | Use | |---|---| | **SELECT** | Choose which columns/fields to return | | **FROM** | Specify the table(s) | | **WHERE** | Filter which rows are returned (AND / OR, LIKE 'A%') | | **JOIN ... ON** | Combine columns from related tables on a matching key | | **ORDER BY** | Sort results (ASC / DESC) | | **INSERT INTO ... VALUES** | Add a new record | | **UPDATE ... SET** | Change existing records | | **DELETE FROM** | Remove records | | **CREATE TABLE** | Define a new table |

Order: **SELECT ... FROM ... WHERE ... ORDER BY .**

**Transactions & ACID** — a transaction = one logical operation; all steps succeed or all fail. | Property | Meaning | |---|---| | **Atomicity** | All-or-nothing; failure → roll back, no partial change | | **Consistency** | Moves DB from one valid state to another (keeps constraints) | | **Isolation** | Concurrent transactions don't interfere | | **Durability** | Once committed, permanently saved (survives failure) |

- **Record locking:** lock a record during a transaction so others can't edit it → prevents **lost update problem**; risk of **deadlock**.
- **Redundancy (good):** duplicate data/hardware (backups, RAID, failover) for reliability — different from *data redundancy* (bad, removed by normalisation).

---

### 1.3.3 Networks

- **LAN:** small area, single site, one organisation owns hardware, high speed. **WAN:** large area (cities/countries), uses third-party infrastructure; internet = largest WAN.

| Client-server                       | Peer-to-peer (P2P)                |
|-------------------------------------|-----------------------------------|
| Server provides services to clients | All peers equal (client & server) |
| Central security/backup             | Decentralised                     |
| Server = single point of failure    | No single point of failure        |
| Scales well, costlier               | Cheaper, harder to manage/secure  |

**Internet structure:** *backbone* (high-capacity routes); *ISP* (provides access); *IP address* (uniquely identifies device; IPv4 32-bit, IPv6 128-bit); *DNS* (translates domain names → IP addresses, hierarchical & cached).

**Topologies:** **Star** — devices to central switch/hub; one cable fault isolated, but central node = single point of failure. **Mesh** — multiple paths between nodes; resilient/fault-tolerant, no single point of failure, but expensive (wired).

| Packet switching                              | Circuit switching                                  |
|-----------------------------------------------|----------------------------------------------------|
| Data split into packets, routed independently | Dedicated path for whole communication             |
| Reassembled at destination                    | One fixed reserved path                            |
| Efficient, resilient (used by internet)       | Guaranteed bandwidth, wastes idle capacity (phone) |

- **Protocol** = rules for communication. **Standard** = agreed spec ensuring interoperability.

**TCP/IP stack** (data goes *down* when sending, *up* when receiving; each layer adds/removes a header) | Layer | Role | Protocols | |---|---|---| | Application | Services for user apps; prepares data | HTTP, HTTPS, FTP, SMTP, POP3, IMAP, DNS | | Transport | Splits into packets, end-to-end delivery, reassembly, ports | TCP, UDP | | Network/Internet | Adds source/destination **IP**, routes packets | IP | | Link/Data link | Physical connection, adds **MAC** address | Ethernet, Wi-Fi |

**Protocols:** HTTP = web pages; HTTPS = encrypted HTTP (TLS/SSL); FTP = file transfer; SMTP = *sending* mail; POP3 = retrieve mail (downloads & removes); IMAP = retrieve mail (stays on server, syncs).

**Hardware:** NIC (connects device, holds MAC); MAC address (unique 48-bit hardware address); **switch** (connects devices *within* a LAN, uses MAC); **router** (connects *different* networks, uses IP); WAP (wireless access to wired network).

**Security:** *firewall* (controls traffic by rules / packet filtering); *proxy* (intermediary — hides IP, caches, filters/logs); *encryption* (protects data in transit).

---

### 1.3.4 Web Technologies

---

| Tech       | Role                                                                                                    |
|------------|---------------------------------------------------------------------------------------------------------|
| HTML       | Structure & content (headings, paragraphs, links, images)                                               |
| CSS        | Presentation/style (colour, font, layout); selectors target tag, <code>.class</code> , <code>#id</code> |
| JavaScript | Interactivity/behaviour; usually client-side (events, validation, dynamic updates)                      |

- **Search indexing:** web crawler/spider follows links, reads pages; engine builds an **index** mapping keywords → pages for fast retrieval, then ranks by relevance.
- **PageRank:** ranks pages by *importance* from incoming links. Each link = a "vote"; links from high-rank pages worth more; a page divides its rank across its outgoing links; **damping factor** (~0.85) models a random surfer; **iterative** until values **converge**.

|             | Client-side                       | Server-side                   |
|-------------|-----------------------------------|-------------------------------|
| Runs on     | User's browser                    | Web server                    |
| Speed       | Fast, immediate                   | Slower (round trip)           |
| Security    | Insecure (can be viewed/bypassed) | Secure (hidden)               |
| Server load | Reduced                           | Increased                     |
| Example     | JS form validation                | DB query, login/payment check |

## Must-know definitions

| Term                              | Definition                                                                     |
|-----------------------------------|--------------------------------------------------------------------------------|
| Lossless / Lossy                  | No permanent data loss (recoverable) / permanent removal (not recoverable)     |
| RLE                               | Lossless: replaces runs of repeated values with value + count                  |
| Dictionary compression            | Lossless: stores repeated patterns once, references them                       |
| Symmetric / Asymmetric            | Same key for both / public + private key pair                                  |
| Hashing                           | One-way function producing a fixed-length hash                                 |
| Primary / Foreign / Composite key | Uniquely IDs record / PK of another table linking tables / PK of 2+ attributes |
| Normalisation                     | Organising data to reduce redundancy & remove anomalies                        |
| Referential integrity             | Foreign keys must reference a valid existing primary key                       |
| Transaction / ACID                | All-or-nothing operation / Atomicity, Consistency, Isolation, Durability       |
| Record locking                    | Locking a record during a transaction to stop concurrent edits                 |
| LAN / WAN                         | Small single-site network / large geographically spread network                |
| Packet / Circuit switching        | Packets routed independently / dedicated reserved path                         |
| Protocol                          | Set of rules governing communication                                           |
| DNS                               | Translates domain names into IP addresses                                      |
| MAC address                       | Unique hardware address of a NIC (link layer)                                  |
| PageRank                          | Ranks pages by importance from incoming links                                  |

## ★ Top exam reminders

- **Lossy can NOT be reversed** — "reduced quality" ≠ "recoverable". RLE does **not** always shrink files (only with long runs).
- **Hashing ≠ encryption** — hashing is one-way; encryption is reversible with a key. Passwords are **hashed**, not encrypted.
- A **foreign key** is the **primary key of another table** — always name which table. For normalisation, name the dependency removed at each stage (repeating group → 1NF, partial → 2NF, transitive → 3NF) and explain *why* (remove redundancy/anomalies).
- **Switch** = devices within a LAN (MAC); **router** = different networks (IP). Don't swap them. Two meanings of "redundancy": data (bad) vs hardware/backup (good).

- **Client-side validation is NOT secure** — it can be bypassed; security-critical checks (login, payment) must also run **server-side**.